数字逻辑与处理器基础·第一次大作业·实验报告

无 48 吴泓锦 2024010654

〇、承诺书

第零部分　承诺书

在提交的实验报告首页，应附上已填写完成的承诺书截图。承诺书内容如下：

本人郑重承诺：已按照大作业文档中“提交要求”部分的说明，对所有相关文件进行整理，并按规定格式进行压缩打包提交。若因文件格式、命名或提交方式等不符合要求而导致丢分，本人愿承担全部责任。

同时，作业中如涉及使用人工智能（AI）生成或辅助完成的内容，均已在对应题目处明确标注所使用的 AI 提供商及模型名称，并在实验报告的附录中以截图形式提供与 AI 的完整对话记录，确保内容真实、来源清晰、过程可追溯。

承诺人：吴泓锦

日　期：2026.6.7

一、有符号整数比较器设计

1-1. 组合逻辑设计

1-1-1. 设计思路

整体思路：

在这个模块中，由于不能直接使用不等号做判断，我们只能按位比较。同时，由于 2 补码的最高位（num_x[WIDTH-1]，x=a/b）代表其正负，因此可以分以下三种情况：

（1）num_a[WIDTH-1]=0 且 num_b[WIDTH-1]=1，a 正 b 负，输出 greater_than=1；

（2）num_a[WIDTH-1]=1 且 num_b[WIDTH-1]=0，a 负 b 正，输出 greater_than=0；

```
22        if ((!num_a[WIDTH-1]) && (num_b[WIDTH-1])) begin
23            greater_than_reg = 1'b1;
24        end
25
26        else if ((num_a[WIDTH-1]) && (!num_b[WIDTH-1])) begin
27            greater_than_reg = 1'b0;
28        end
```

（3）num_a[WIDTH-1]=num_b[WIDTH-1]=0 或 1，二者同正或同负。此时只需比较除去最高位的后 WIDTH-1 位组成的二进制数的大小（例如十进制 5>3，相应补码$(0101)_2$ 的后三位大于$(0011)_2$；十进制-1>-2，相应补码$(1111)_2$ 的后三位大于$(1110)_2$）。因此，使用 repeat 循环，从第 WIDTH-2 位开始向下逐位比较。标记变量 higher_bits_equal 初值为 1，作为是否继续逐位比较的入口条件。如遇到某一位上 num_a 和 num_b 不相等，则根据该位直接输出 a 和 b 的大小关系结果，并且将 higher_bits_equal 置 0，不再进行后续位的比较。

```
30        else begin
31            i = WIDTH - 2;
32            repeat (WIDTH - 1) begin
33            if (higher_bits_equal) begin
34                    if ((num_a[i]) && (!num_b[i])) begin
35                        greater_than_reg = 1'b1;
36                        higher_bits_equal = 1'b0;
37                    end
38                    else if ((!num_a[i]) && (num_b[i])) begin
39                        greater_than_reg = 1'b0;
40                        higher_bits_equal = 1'b0;
41                    end
42                end
43                i = i - 1;
44            end
45        end
```

新定义变量说明：

（1）reg greater_than_reg：由于 wire 型变量不能在过程块语句中被赋值，因此再定义一个 reg 型变量用于临时存储结果，并且在过程块语句外用 assign 语句将 greater_than_reg 的值再赋给 greater_than；

（2）reg higher_bits_equal：见“整体思路”（3）；

（3）integer i：逐位比较循环中的迭代指数，用于指示当前比较的位数。

1-1-2. 仿真结果

仿真原理：

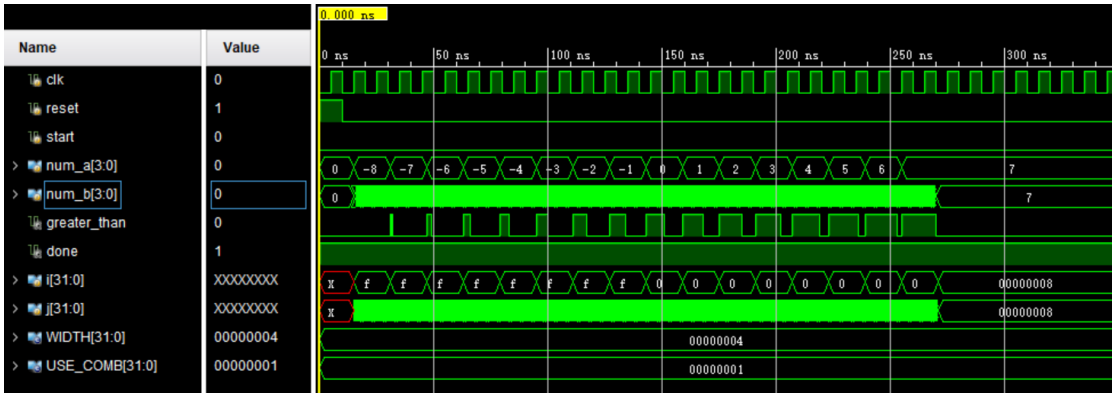
WIDTH=4，自动遍历 4 位补码所有可能的输入组合，检查比较器输出 greater_than 是否等于 Verilog 自带的比较表达式 a>b 的结果。

testbench 中用参数 USE_COMB 控制使用哪个比较器，USE_COMB=1 时例化组合逻辑比较器，USE_COMB=0 时例化时序逻辑比较器。

对于每两个数的比较，结果的正确与否会打印在 Tcl Console 中。

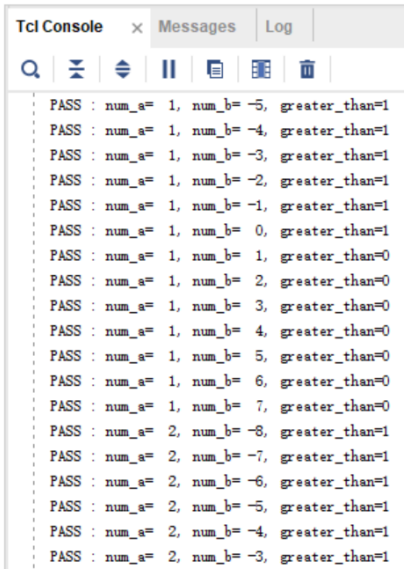
仿真结果：

时序图：



num_a 从-8 到 7 递增，而在 num_a 每个同取值的时间段内，num_b 都从-8 到 7 递增，共测试 16*16=256 种组合。因此，在 num_a 每个同取值的时间段内，greater_than 前面一段时间恒为 1，后面一段时间恒为 0，且随 num_a 增大，greater_than 高电平脉冲越来越宽。

Tcl Console 输出结果（截取部分）：

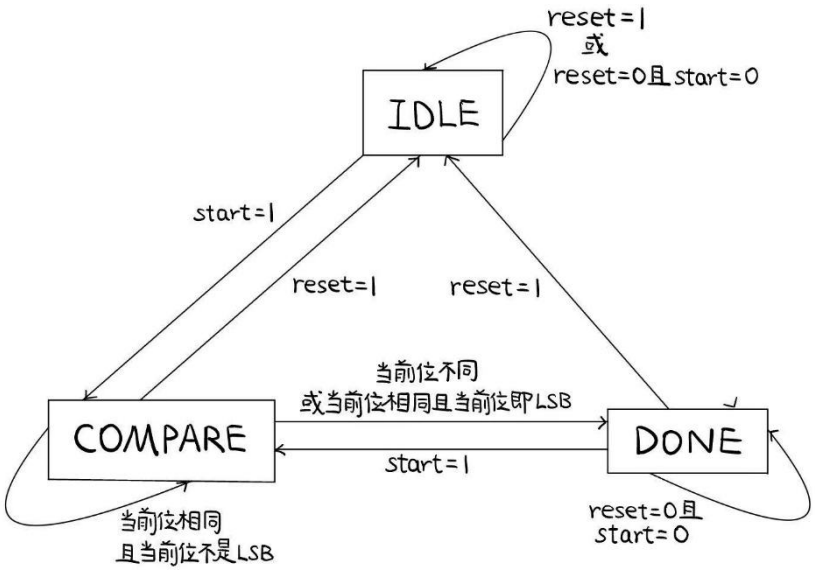


所有比较均输出 PASS，因此可以验证代码正确。

1-2. 时序逻辑设计

1-2-1. 设计思路

状态转移图：



整体思路：

比较的思路与组合逻辑基本相同，也即先比符号位再逐位向下比。在此基础上，结合各使能信号和时序逻辑考虑每个状态应当进行哪些操作、以及状态间的转移逻辑。

各状态功能与转移逻辑：

（0）复位：任何时刻接收到复位信号，转移至 IDLE，清空内部寄存器及输出。

（1）IDLE：空闲状态，等待 start 信号。

- reset=1 或 reset=0 且 start=0 时，依然保持 IDLE 状态；
- reset=0 且 start=1 时，准备比较（下一时钟周期开始比较），锁存 num_a 和 num_b，将 bit_index 置于最高位，转移到 COMPARE 状态。

```
42          // state IDLE: waiting for start signal.
43          IDLE: begin
44              done <= 1'b0;
45              greater_than <= 1'b0;
46
47              if (start) begin
48                  a_reg <= num_a;
49                  b_reg <= num_b;
50                  bit_index <= WIDTH - 1;
51                  state <= COMPARE;
52              end
53              else begin
54                  state <= IDLE;
55              end
56          end
```

（2）COMPARE：逐位比较状态，每个时钟周期比较一位。

- reset=1 时，复位，转移到 IDLE 状态；
- 若当前比较的是最高位（MSB）：如果 MSB 不同，则直接得出结果，直接对 greater_than 相应赋值，done 置为 1，状态转移至 DONE，结束比较；如果 MSB 相同且不是 LSB，则依然保持 COMPARE，比较后面几位；如果 MSB 相同且是 LSB（一般不会出现），则 greater_than 赋值为 0，done 置为 1，转移至 DONE。（见下页左图）

```
// First compare the MSB, namely the sign bit.
if (bit_index == WIDTH - 1) begin
    // Different sign bits:
    // num_a is non-negative and num_b is negative, so num_a > num_b.
    if ((!a_reg[WIDTH-1]) && (b_reg[WIDTH-1])) begin
        greater_than <= 1'b1;
        done <= 1'b1;
        state <= DONE;
    end

    // num_a is negative and num_b is non-negative, so num_a > num_b is false.
    else if ((a_reg[WIDTH-1]) && (!b_reg[WIDTH-1])) begin
        greater_than <= 1'b0;
        done <= 1'b1;
        state <= DONE;
    end

    // Same sign bits, continue to compare the remaining bits.
    else begin
        if (bit_index == 0) begin
            // This only matters when WIDTH = 1.
            greater_than <= 1'b0;
            done <= 1'b1;
            state <= DONE;
        end
        else begin
            bit_index <= bit_index - 1;
            state <= COMPARE;
        end
    end
```

```
// Compare the remaining bits from high to low.
else begin
    if ((a_reg[bit_index]) && (!b_reg[bit_index])) begin
        greater_than <= 1'b1;
        done <= 1'b1;
        state <= DONE;
    end
    else if ((!a_reg[bit_index]) && (b_reg[bit_index])) begin
        greater_than <= 1'b0;
        done <= 1'b1;
        state <= DONE;
    end
    else begin
        // If the current bit is the LSB, all bits are equal.
        if (!bit_index) begin
            greater_than <= 1'b0;
            done <= 1'b1;
            state <= DONE;
        end
        else begin
            bit_index <= bit_index - 1;
            state <= COMPARE;
        end
    end
end
```

- 若比较的不是最高位：如果所比较的当前位不同，直接相应对 greater_than 赋值为 0/1，标记 done 为 1，转移到 DONE；如果所比较的当前位相同且不是 LSB，则 bit_index 减 1，等待下一周期再比较下一位；如果所比较的当前位相同且当前位已达 LSB，则直接对 greater_than 赋值为 0，done 标记为 1，转移到 DONE。（见上方右图）

综上所述，COMPARE 的非复位转移逻辑可以概括为：如果当前位不同，则转移至 DONE；如果当前位相同，且当前位即为最低位，则转移至 DONE；如果当前位相同，且当前位不是最低位，则保持 COMPARE 继续比较更低位。

（3）DONE：比较完成状态，done=1，greater_than 有效。

- reset=1 时，复位，转移到 IDLE；
- reset=0 且 start=0 时，保持 DONE；
- reset=0 且 start=1 时，准备比较（下一时钟周期开始比较）。锁存 num_a 和 num_b，将 bit_index 置于最高位，转移到 COMPARE 状态。

```
123     DONE: begin
124         done <= 1'b1;
125
126         if (start) begin
127             done <= 1'b0;
128             greater_than <= 1'b0;
129             a_reg <= num_a;
130             b_reg <= num_b;
131             bit_index <= WIDTH - 1;
132             state <= COMPARE;
133         end
134         else begin
135             state <= DONE;
136         end
137     end
```

新定义变量说明：

（1）reg [1:0] state：标记当前状态；

（2）integer bit_index：逐位比较循环中的迭代指数，用于指示当前比较的位数；

（3）reg [WIDTH-1:0] a_reg、reg [WIDTH-1:0] b_reg：在 start 跳变为 1（即开始比较）的第一个时钟周期锁存输入的 num_a 和 num_b，避免后续外部输入变化影响比较。

1-2-2. 仿真结果

仿真原理：

在组合逻辑的测试原理的基础上增加了时序控制，给定每一轮的 num_a 和 num_b 之后产生 start 启动脉冲，等待 WIDTH+1 个周期，之后检查 done 是否为 1。若 done 不为 1，证明不满足性能要求；若 done 为 1，再检查 greater_than 输出是否正确。

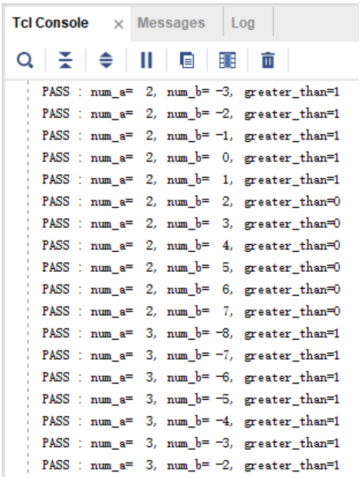
仿真结果：

时序图：



可以观察到随着 num_a 增大，greater_than 高电平在 num_a 取值恒定的时间段内的时间越来越长（与组合逻辑结果类似），说明比它小的数越来越多，所得时序图与预期相符。

Tcl Console 输出结果（截取部分）：



所有比较均输出 PASS，因此可以验证代码正确。

1-3. 基于国产大模型 AI 的 Verilog 编写

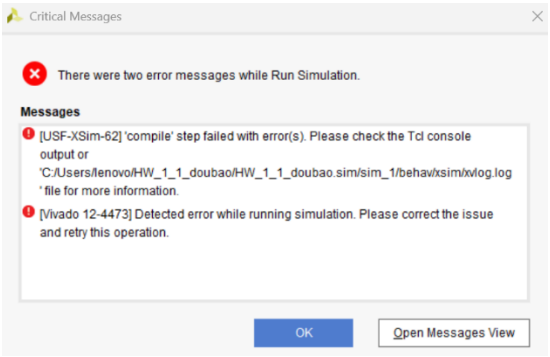
1-3-1-1. 豆包组合逻辑（“豆包 2.0-专家”模型）

需要人工修改的问题：

（1）代码模板中，greater_than 变量为 wire 型，不能直接在过程块语句中赋值，但其生成的代码中直接写成了：

```
34          // 最终结果：符号位不同时用符号判断，相同时用逐位比较结果
35          greater_than = sign_case | (~sign_different & same_sign_greater);
```

导致 Vivado 直接报错，无法编译：

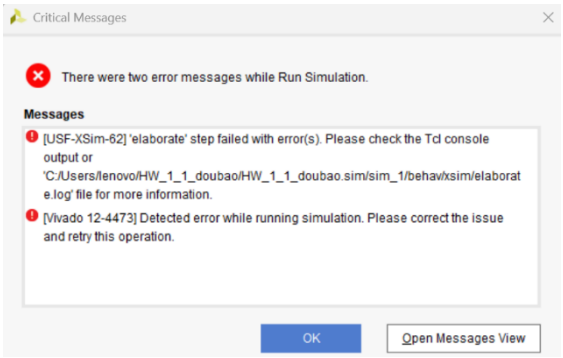


因此，需要额外定义一个 reg 型变量 result 用于在 always 语句中赋值，即临时存储 greater_than 的值，再在 always 语句外部用 assign 语句将 result 的值赋给 greater_than。

（2）代码中 for 循环里使用了 break 语句做退出

```
29          // 从最高位到最低位逐位比较
30          for (i = WIDTH-1; i >= 0; i = i - 1) begin
31              if (num_a[i] != num_b[i]) begin
32                  // 找到第一个不同的位，该位为1的数更大（补码特性）
33                  same_sign_greater = num_a[i];
34                  break;
35              end
36          end
```

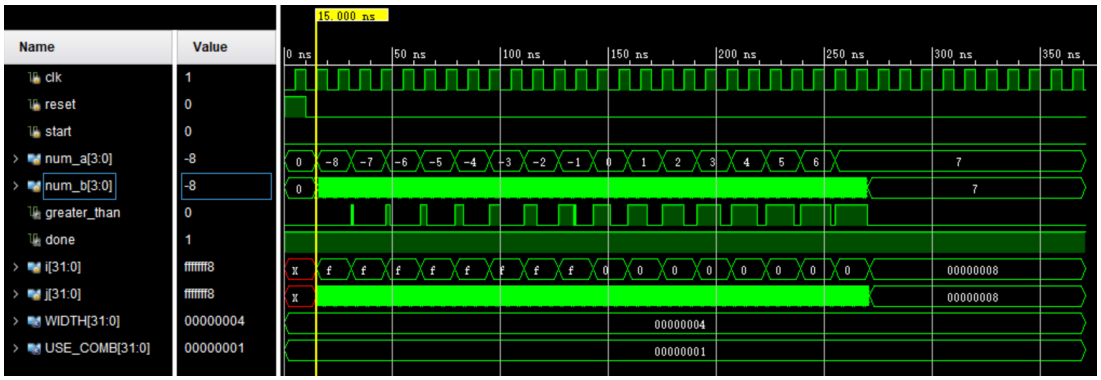
这在传统 Verilog 中不符合语法要求，因此 Vivado 报错，在 elaborate 阶段产生错误：



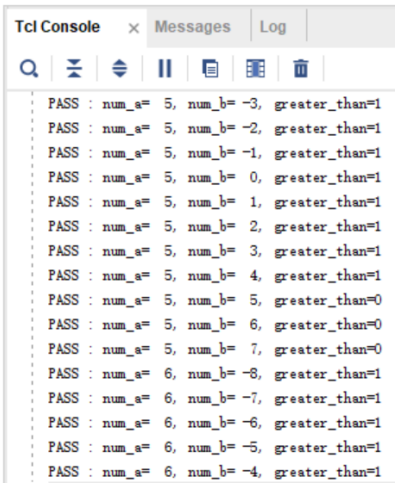
需要类似前面 1-1-1 中定义一个标志变量，例如 reg higher_bits_equal，用来标示已经比较的几个高位是否已经可以判断出结果，从而作为控制循环是否进行的条件，而不是“简单粗暴”地用 break 跳出循环。

仿真结果：

时序图：



Tcl Console 输出结果（截取部分）：



如图可知，经过修改后，仿真可以正常运行，且运行结果正确，全部输出 PASS。

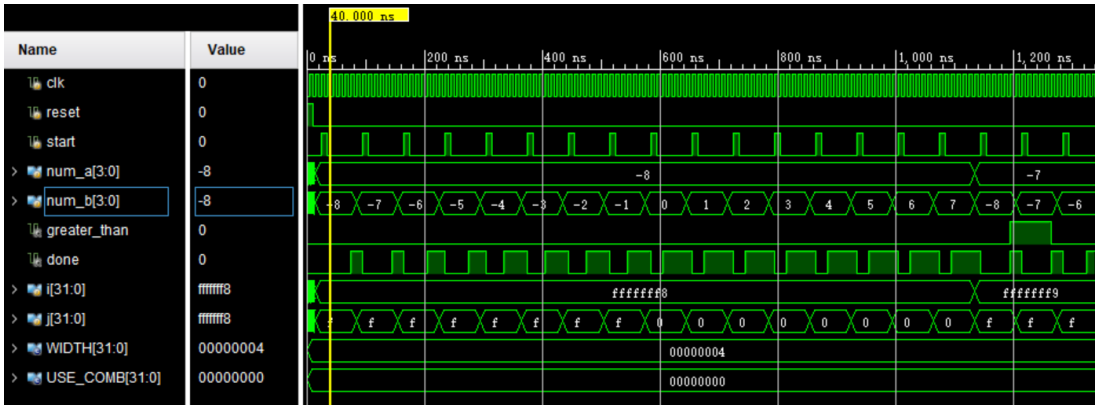
1-3-1-2. 豆包时序逻辑（“豆包 2.0-专家”模型）

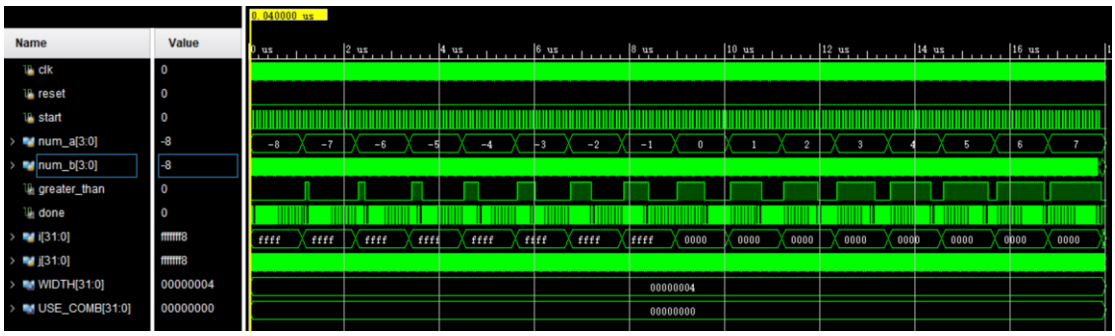
需要人工修改的问题：

无，可以正常仿真运行且仿真结果正确。

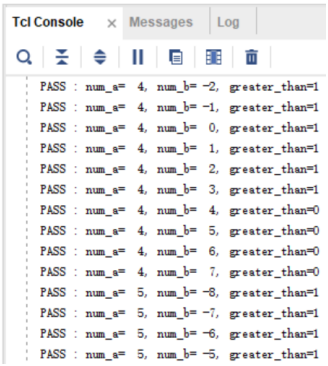
仿真结果：

时序图：





Tcl Console 输出结果（截取部分）：



如图可知，仿真可以正常运行，且运行结果正确，全部输出 PASS。

1-3-1-3. 豆包代码分析

（需要指出，这里使用的是“豆包 2.0-专家”模型。）

代码规范性：

代码整体端口定义、参数 WIDTH 设置与实验模板一致，变量的定义和语句的撰写整体清晰，注释也比较完整，能够体现各信号的基本含义。但是，其组合逻辑代码存在一定的语法问题（见 1-3-1-1），语法规范性不足，有一定“提升空间”。

功能正确性：

组合逻辑代码存在语法错误，无法正常仿真，修改后可以正常仿真并且结果正确。时序逻辑代码整体没有大问题，可以正常仿真且结果正确，可以实现时序触发和比较功能。

设计思路贴合度：

总体贴合核心要求，采用符号位判断和逐位比较的方法，符合手动实现比较器的设计思路。组合逻辑部分体现了按位比较的底层逻辑；时序逻辑部分使用三状态 FSM，每个周期比较一位，较好体现了时序化、结构化的硬件设计思想，也满足实验的基本要求。

可仿真性：

此部分略，根据前面内容可知其撰写的代码存在一定的语法问题，无法直接用于仿真，必须经过人工修改（具体问题见 1-3-1-1）。

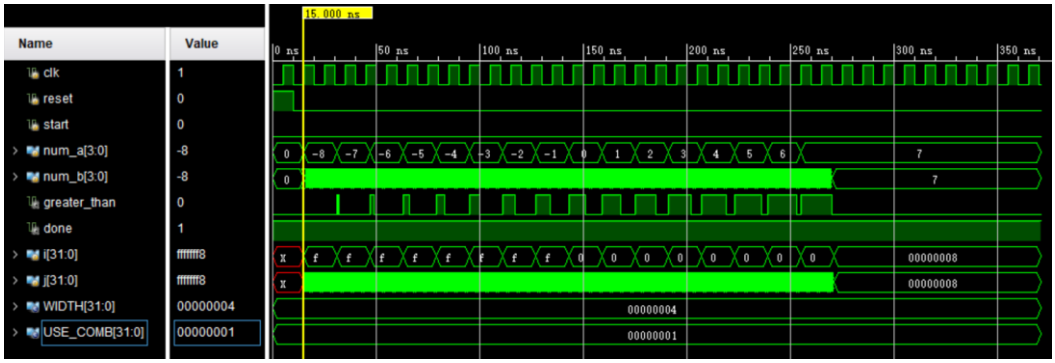
1-3-2-1. Kimi 组合逻辑（“K2.6-思考”模型）

需要人工修改的问题：

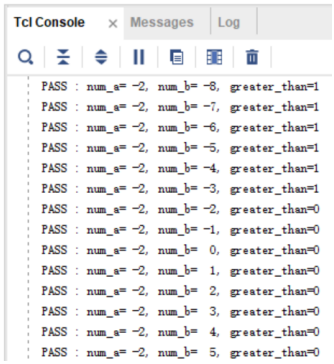
无，可以正常仿真运行且仿真结果正确。

仿真结果：

时序图：



Tcl Console 输出结果（截取部分）：



如图可知，仿真可以正常运行，且运行结果正确，全部输出 PASS。

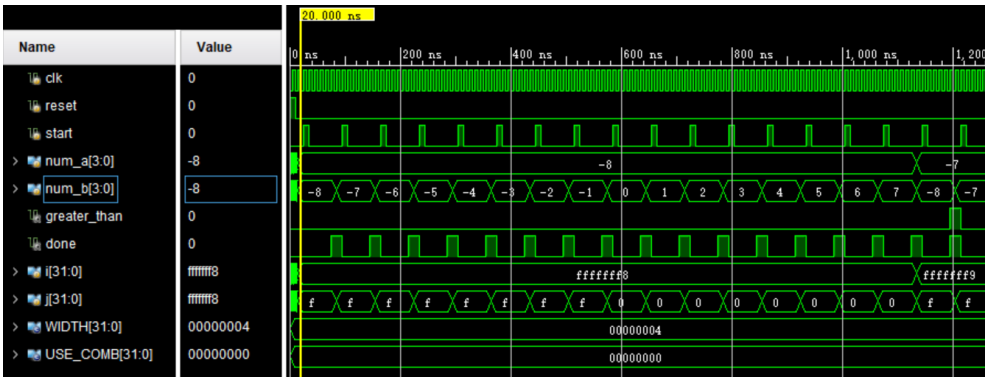
1-3-2-2. Kimi 时序逻辑（“K2.6-思考”模型）

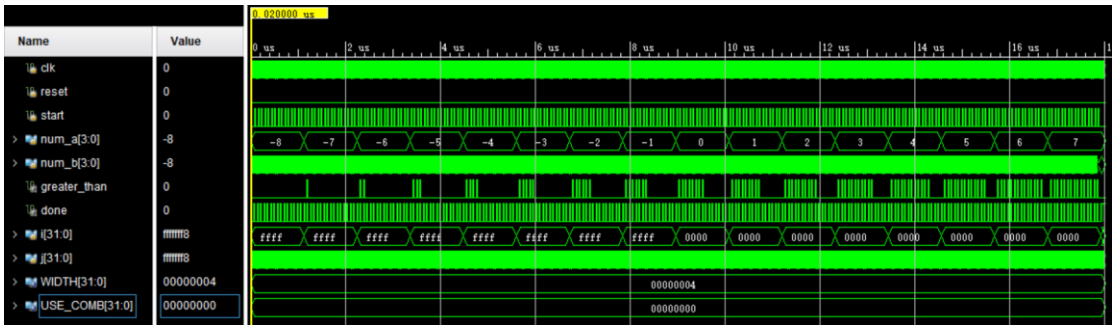
需要人工修改的问题：

无，可以正常仿真运行且仿真结果正确。

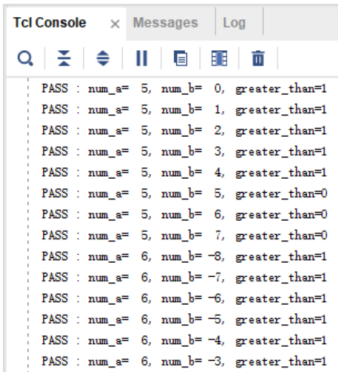
仿真结果：

时序图：





Tcl Console 输出结果（截取部分）：



如图可知，仿真可以正常运行，且运行结果正确，全部输出 PASS。

1-3-2-3. Kimi 代码分析

（需要指出，这里使用的是“K2.6-思考”模型。）

代码规范性：

代码端口定义、参数 WIDTH 设置均与模板保持一致，内部变量也使用参数化位宽，具备一定通用性。代码注释较完整，结构清晰，语法正确，整体符合 Verilog 设计规范。

功能正确性：

两段代码都可以正常仿真且结果正确，组合逻辑代码可以正确实现有符号数比较功能，时序逻辑代码可以正确实现时序触发与比较功能。

设计思路贴合度：

较好地贴合了实验 1-1 和 1-2 的要求，采用符号位判断和逐位比较的方法。组合逻辑代码先判断符号位是否相同，再在符号相同时从高位到低位逐位比较低位部分，可以正确实现补码有符号数比较。时序逻辑代码能够在 start 到来时锁存输入，通过 COMPARE 状态逐位比较，并在 DONE 状态保持 done=1 和输出结果，体现了把组合比较过程时序化的硬件设计思想，符合实验对底层逻辑实现的要求。

可仿真性：

两段代码都可以正常仿真且结果正确。

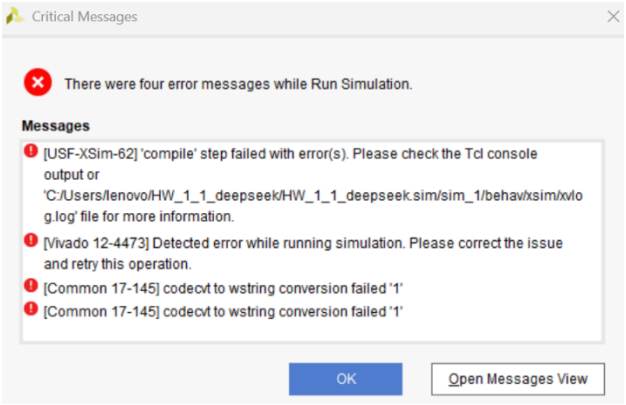
1-3-3-1. DeepSeek 组合逻辑（“DeepSeek-V4-Flash”模型）

需要人工修改的问题：

DeepSeek 比豆包“强一点”，它定义了一个 reg 型变量 gt_tmp 临时存储 greater_than 的值。但是，它依然在 always 语句内对 greater_than 赋值：

```
46        greater_than = gt_tmp;
```

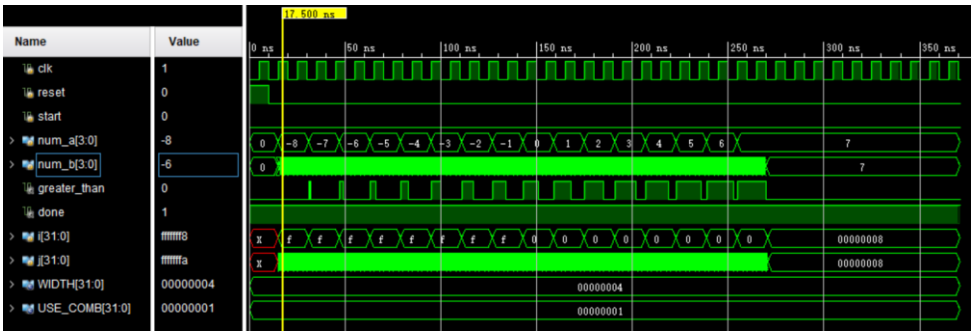
从而导致 Vivado 报错，无法编译：



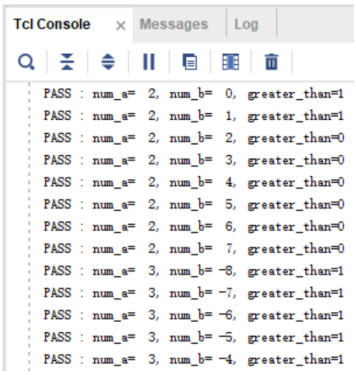
因此，需要将“greater_than = gt_tmp”这一语句移至 always 语句块外面，并且在前面加上“assign”。

仿真结果：

时序图：



Tcl Console 输出结果（截取部分）：

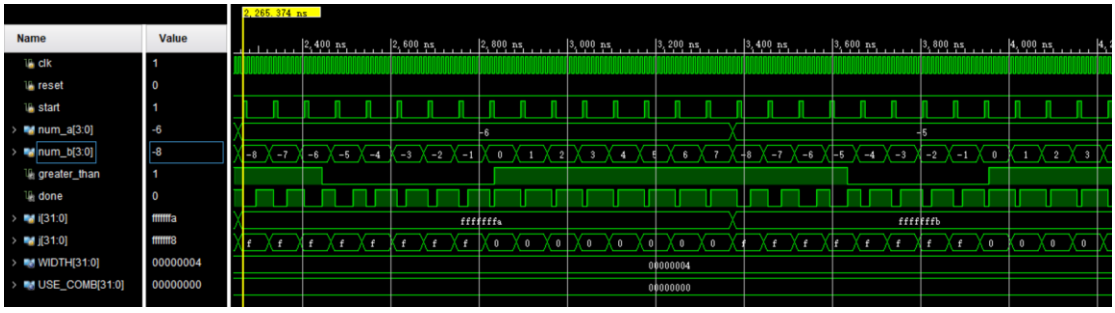


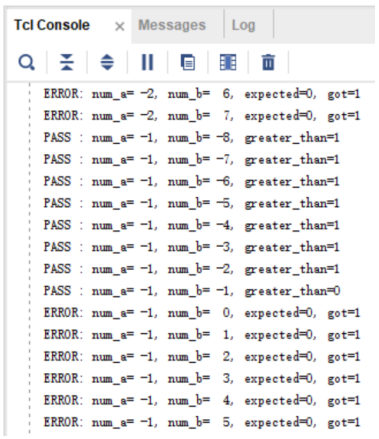
如图可知，经过修改后，仿真可以正常运行，且运行结果正确，全部输出 PASS。

1-3-3-2. DeepSeek 时序逻辑（“DeepSeek-V4-Flash”模型）

需要人工修改的问题：

没有语法错误，可以正常仿真。但是，其仿真结果是错误的：





当 num_a 和 num_b 同号时，输出结果正确；二者异号时，输出结果错误。

分析代码，可以得知它把补码有符号数错误地当成了无符号数进行比较，直接从最高位开始逐位比较：

```
48          COMPARE: begin
49              // 若在比较过程中收到新的 start，立即中断并重新开始
50              if (start) begin
51                  state    <= COMPARE;
52                  a_reg    <= num_a;
53                  b_reg    <= num_b;
54                  bit_idx  <= WIDTH - 1;
55                  done     <= 1'b0;
56                  // greater_than 稍后更新
57              end else begin
58                  // 比较当前位 (a_reg[bit_idx] 与 b_reg[bit_idx])
59                  if (a_reg[bit_idx] && ~b_reg[bit_idx]) begin
60                      // 当前位 A=1, B=0 => A > B
61                      greater_than <= 1'b1;
62                      done         <= 1'b1;
63                      state        <= DONE;
64                  end else if (~a_reg[bit_idx] && b_reg[bit_idx]) begin
65                      // 当前位 A=0, B=1 => A < B
66                      greater_than <= 1'b0;
67                      done         <= 1'b1;
68                      state        <= DONE;
```

如图，可以看出其开始比较的位数，也即 bit_idx 的初值是 WIDTH-1，即最高位，因此产生了错误。（但是我是在同一个对话中让它生成的组合逻辑和时序逻辑这两段代码，组合逻辑代码并没有出现这个错误，不知原因在哪。）

因此，我们需要重新修改比较的逻辑，bit_idx 初值设置为 WIDTH-2。在一次比较进程中，先判断第 WIDTH-1 位是否相等，如果不相等，直接输出 greater_than 的结果，置 done 为 1，将状态转移为 DONE；如果最高位不相等，则转移状态至 COMPARE 逐位比较。

仿真结果：

起初错误的仿真结果已在“需要人工修改的问题”部分展示，修改正确之后的代码的仿真结果与先前两个大模型生成代码的仿真结果完全相同，不再展示。

1-3-3-3. DeepSeek 代码分析

（需要指出，这里使用的是“DeepSeek-V4-Flash”模型。）

代码规范性：

代码整体端口定义和参数 WIDTH 设置与实验模板一致，注释较完整，代码可读性较好。但是，组合逻辑代码存在语法错误，与豆包的错误类似，都是把 wire 型变量放在了 always 语句内进行赋值，语法规范性不足，同样有一定“提升空间”。

功能正确性：

组合逻辑代码在修正 wire 赋值问题后可以正常仿真，且仿真结果正确，能够实现有符号数比较。时序逻辑代码虽然可以通过编译并运行仿真，但结果错误：它直接从最高位开始按无符号方式逐位比较，把符号位 1 误认为“大于”符号位 0，导致负数与非负数比较时结果全部颠倒，仿真结果错误。

设计思路贴合度：

组合逻辑代码思路较贴合实验要求，采用“先判断符号位，符号相同再逐位比较”的方法，没有直接使用大小比较运算符。时序逻辑代码虽然使用了 FSM 和逐位比较，形式上符合时序化设计思路，但没有正确区分符号位和普通数值位，本质上实现成了无符号逐位比较，因此未能完整体现有符号补码比较器的核心设计原理。

可仿真性：

组合逻辑代码存在语法错误无法仿真，修改后可以正常进行仿真且仿真结果正确；时序逻辑代码可以正常仿真，但是仿真结果错误。

（三种大模型的整体比较请见下页）

1-3-4. 三种大模型代码整体分析

代码规范性：

三者生成的代码在模块名、端口定义、WIDTH 参数设置方面都基本符合实验模板要求，注释也较完整，整体可读性较好。相比之下，Kimi 表现最好，组合逻辑和时序逻辑都能保持较规范的 Verilog 写法，语法最准确；豆包和 DeepSeek 表现较差，都在组合逻辑代码中犯了同类错误，即将变量 output wire greater_than 放在 always 块中赋值。豆包还使用了 break，在传统 Verilog 中兼容性较差。

功能正确性：

三者的组合逻辑设计思路都基本正确，修改语法问题后都能实现有符号数比较。时序逻辑方面，Kimi 和豆包均能通过仿真测试，能够实现正常的时序触发与比较功能；但 DeepSeek 的时序逻辑虽然能编译运行，仿真结果却错误，因为它把符号位也按普通数值位处理，导致负数与非负数比较时结果颠倒。因此从功能正确性看，Kimi 和豆包表现较好，DeepSeek 的时序逻辑存在明显功能错误，表现较差。

设计思路贴合度：

三者都没有直接使用<、>、<=、>=或加减法比较大小，整体上符合实验要求的“手动实现比较器”思路。Kimi 的设计最贴合实验要求：组合逻辑中明确区分符号位和数值位，时序逻辑中也使用状态机逐位比较，较好体现了底层硬件实现思想。豆包的思路也比较接近，组合逻辑和时序逻辑都体现了符号判断与逐位比较。DeepSeek 的组合逻辑贴合度较好，但时序逻辑没有正确处理补码符号位，本质上实现成了无符号逐位比较，说明其对“有符号比较”的核心原理理解不充分。

可仿真性：

Kimi 的两份代码均可直接通过仿真，表现最好。豆包的时序逻辑可以直接仿真且结果正确，但组合逻辑原始代码需修改 wire 过程赋值和 break 语句后才能正常使用。DeepSeek 的组合逻辑也需要将 wire 赋值的问题修改后才能编译；其时序逻辑虽然无需语法修改即可运行，但仿真结果错误，需要进一步修改符号位处理逻辑后才能满足实验要求。总体而言，Kimi 的代码可用性最高，豆包需要小幅语法修正，DeepSeek 则既有语法问题，也有时序逻辑功能问题。

二、有符号数据最值输出模块设计

2-1. 设计思路

整体思路：

整个程序可以看成一个两级比较网络：

第一级：例化两个比较器 comp_ab 和 comp_cd，分别比较（num_a，num_b）和（num_c，num_d），输出两组比较的结果。同时，定义变量 max_ab、min_ab、max_cd、min_cd 分别存储两组比较各自的最大值和最小值，用 assign 语句和三目表达式赋值。

```
23    // Comparator 1: num_a vs num_b
24    signed_compare_comb #(
25        .WIDTH(WIDTH)
26    ) comp_ab (
27        .num_a(num_a),
28        .num_b(num_b),
29        .greater_than(gt_ab)
30    );
31
32    // Comparator 2: num_c vs num_d
33    signed_compare_comb #(
34        .WIDTH(WIDTH)
35    ) comp_cd (
36        .num_a(num_c),
37        .num_b(num_d),
38        .greater_than(gt_cd)
39    );
```

```
41    // Select max and min from the first pair (a, b)
42    assign max_ab = gt_ab ? num_a : num_b;
43    assign min_ab = gt_ab ? num_b : num_a;
44
45    // Select max and min from the second pair (c, d)
46    assign max_cd = gt_cd ? num_c : num_d;
47    assign min_cd = gt_cd ? num_d : num_c;
72    always @(posedge clk or posedge reset) begin
73        // TODO: Implement your Signed MM4 sequential logic here
74        if (reset) begin
75            Max <= {WIDTH{1'b0}};
76            Min <= {WIDTH{1'b0}};
77        end
78        else if (input_valid) begin
79            Max <= max_result;
80            Min <= min_result;
81        end
82        // When input_valid is 0, Max and Min hold their previous values
```

第二级：用第一级的比较结果再进行比较，例化比较器 comp_max，比较 max_ab 和 max_cd，选出全局最大者；例化比较器 comp_min，比较 min_ab 和 min_cd，选出全局最小者。再定义存储结果的变量 max_result 和 min_result，用 assign 语句和三目表达式赋值。

```
49    // Comparator 3: max_ab vs max_cd for final Max
50    signed_compare_comb #(
51        .WIDTH(WIDTH)
52    ) comp_max (
53        .num_a(max_ab),
54        .num_b(max_cd),
55        .greater_than(gt_max)
56    );
57
58    // Comparator 4: min_ab vs min_cd for final Min
59    signed_compare_comb #(
60        .WIDTH(WIDTH)
61    ) comp_min (
62        .num_a(min_ab),
63        .num_b(min_cd),
64        .greater_than(gt_min)
65    );
```

```
67    // Final selection for Max and Min
68    assign max_result = gt_max ? max_ab : max_cd;
69    assign min_result = gt_min ? min_cd : min_ab;
```

最后，时序部分（always 语句块），在 reset 高电平时将输出 Max 和 Min 置零，在 reset 低电平且 input_valid 有效时对 Max 和 Min 赋值即可。（这里 reset 低电平且 input_valid 低电平的逻辑分支缺省，起到了锁存作用）（代码见右上方图 Line72~81）

新定义变量说明：（（2）请见“整体思路”中的说明）

（1）wire gt_ab, gt_cd, gt_max, gt_min：四个比较器的输出结果（0 或 1）；

（2）wire [WIDTH-1:0] max_ab, min_ab, max_cd, min_cd, max_result, min_result：略。

2-2. 仿真结果

仿真原理：

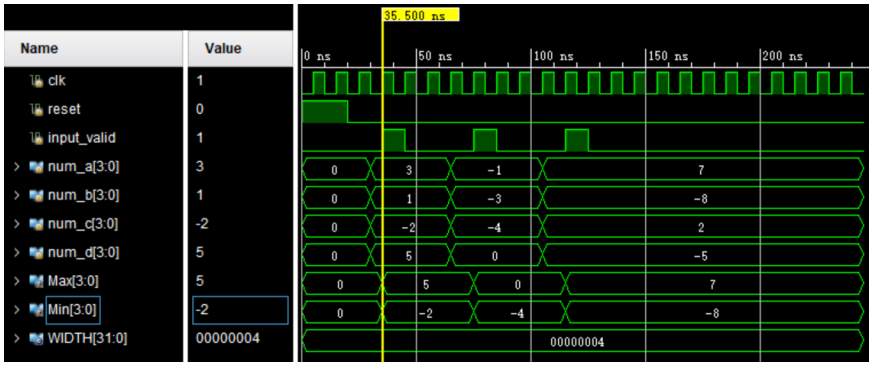
先整体复位清零，再测试三组输入值。

在每一组测试中，先通过 testbench 中的函数 max4 和 min4 计算预期结果；再在下一个 clk 上升沿置高 input_valid，持续 1 个时钟周期，然后置低；最后再等待 2 个时钟周期，让输出稳定，比较模块输出和预期结果。

每组数测试后，结果的正确与否会打印在 Tcl Console 中。

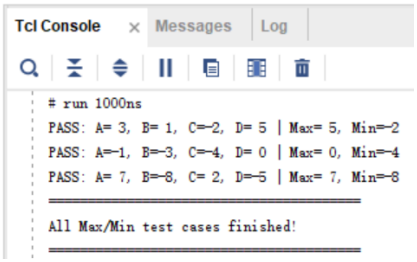
仿真结果：

时序图：



如图，可以观察到三组输入数据测试的结果均正确，满足逻辑和时序要求：程序先复位，每次测试时，先更新输入，在随后的 clk 上升沿置高 input_valid，Max 和 Min 也在 clk 上升沿更新，input_valid 高电平持续 1 个周期后置低，Max 和 Min 在下一次 input_valid 置高前保持不变。

Tcl Console 输出结果：



三组测试均输出 PASS，因此可以验证代码正确。

三、有符号数据最值输出模块设计

3-1. 设计思路

整体思路：

例化两个组合逻辑比较器 comp_data_vs_max 和 comp_min_vs_data，用于比较当前输入的数据 data 和先前的 Max、Min。

需要说明，程序中需要定义一个标志变量 initialized，用于标记当前时钟周期所处理的数据是不是这一轮中的第一个，如果是第一个，则直接把该数据的值作为 Max 和 Min 的值。这是因为，如果不引入该变量，在 start 被置高的时钟周期，Max 和 Min 初始化为 0，如果后续输入的数据为正，则 Min 无法更新，如果后续输入的数据为负，则 Max 无法更新，从而引起错误。

在时序部分中：

- 当 reset=1 时，模块复位，内部的寄存器和变量 Max、Min 和 initialized 清空；
- 当 reset=0 且 start=1 时，新的序列准备输入，把 Max、Min 和 initialized 置零；
- 当 reset=0、start=0 且 input_valid=1 时，说明处于检测模式且数据输入有效，需要检测并更新数据。此时通过 initialized 判断是不是第一个数据。如果是第一个，则直接把该数据的值赋给 Max 和 Min；如果不是第一个，则根据两个比较器的结果判断是否要更新 Max 和 Min；

```
56          // First valid data: directly let Max & Min = current data
57          else if (input_valid) begin
58              if (!initialized) begin
59                  Max <= data;
60                  Min <= data;
61                  initialized <= 1'b1;
62              end
63
64              // Not the first valid data: compare and update
65              else begin
66                  if (gt_data_max) begin
67                      Max <= data;
68                  end
69                  if (gt_min_data) begin
70                      Min <= data;
71                  end
72              end
73          end
```

- 其他情况下，数据输入无效，不更新数据，程序锁存 Max 和 Min。

（由于大部分代码较为常规，此处只展示 reset=0、start=0 且 input_valid=1 时的代码）

这里虽然没有定义 FSM 的各个状态名，但依然通过 reset、start、input_valid 和 initialized 四个控制信号实现了不同状态的分流。

新定义变量说明：

（1）wire gt_data_max, gt_min_data：两个比较器的输出（0 或 1）；

（2）reg initialized：见“整体思路”第二段。

3-2. 仿真结果

仿真原理：

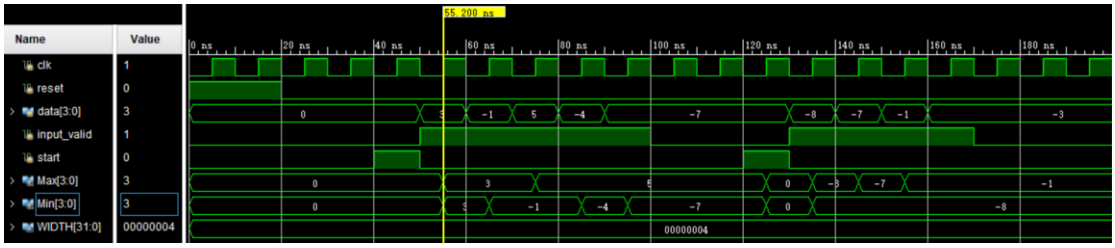
先整体复位清零，再测试两个数据序列。

在测试每个序列前，先给一个单周期 start 脉冲，用于清空各变量并标明检测模式开始。随后拉高 input_valid 表示后续输入有效，每个时钟周期输入一个新数据。序列输入完毕后，input_valid 拉低，调用 check_result 检验实际输出与理论值是否相符。

每个序列检测时，每输入一个新数据，Tcl Console 都会打印一次最新的 Max 和 Min。在每个序列输入完毕后，Tcl Console 也会打印输出最值与理论值是否相符的测试结果。

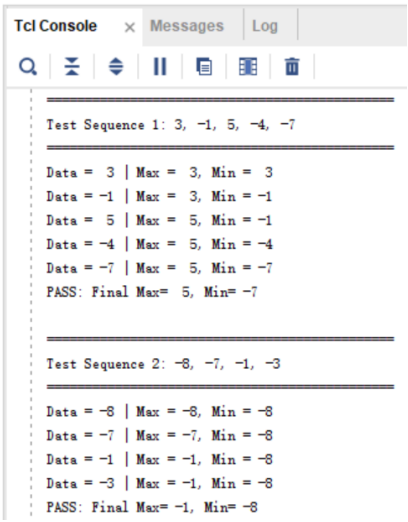
仿真结果：

时序图：



如图，程序先后测试了（3，-1，5，-4，-7）和（-8，-7，-1，-3）两个数据流。序列检测时，每个时钟周期输入一个数，Max 和 Min 在紧跟着的时钟上升沿处更新，每一次更新都符合预期，且各个变量的状态也都符合预期。（图较长，未截全，后面一直维持当前状态至 280ns 结束仿真）

Tcl Console 输出结果：



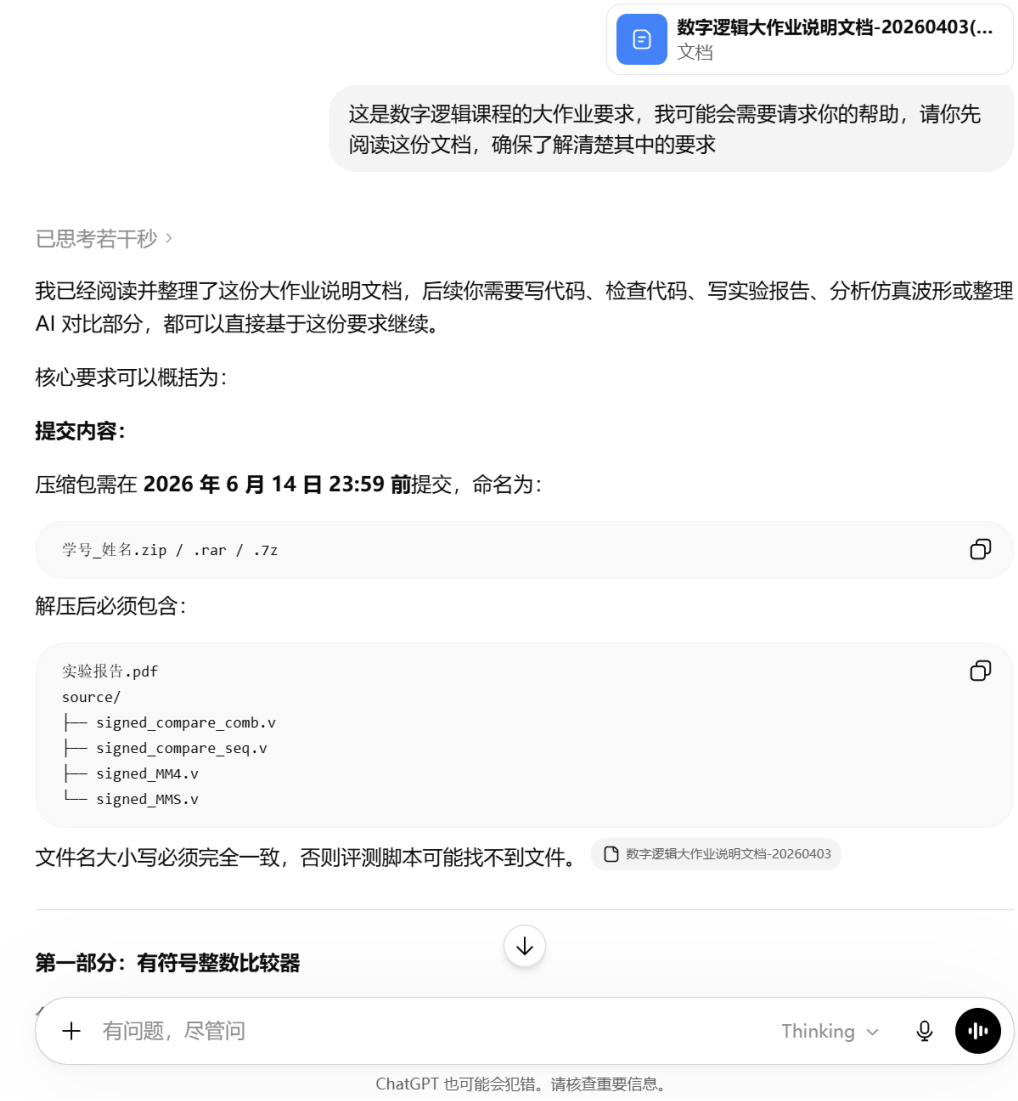
两组测试均输出 PASS，且各个 Max 和 Min 也都正确，因此可以验证代码正确。

附录、与 AI 对话截图

AI 提供商与模型名：OpenAI-GPT-5.5（Thinking 模式）

（不包含在实验 1-3 中所使用的三种国产大模型）

（由于 Verilog 的编写和 Vivado 的使用已在实验课中学过，因此本实验使用 AI 较少）



第一部分：有符号整数比较器

分为组合逻辑和时序逻辑两个模块：

1. `signed_compare_comb.v`

实现有符号数比较：

```verilog
greater_than = 1 表示 num_a > num_b
```

但**禁止直接使用** `<`、`>`、`<=`、`>=` **等比较运算符**，也不能用 `A-B`、`A+5` 这类加减法之后再比较大小，必须体现按位逻辑或逐位比较思想。 数字逻辑大作业说明文档-20260403

2. `signed_compare_seq.v`

基于时序逻辑实现有符号比较器，要求有：

```verilog
clk, reset, start, num_a, num_b
greater_than, done
```

`start` 只保持一个周期；比较完成后 `done` 置高并保持；新的 `start` 或 `reset` 会清零 `done`。并且要求从 `start` 到 `done` 的上升沿间隔不超过 **WIDTH + 1 个周期。** 数字逻辑大作业说明文档-20260403

第一部分 1-3：国产大模型对比

需要选 **三款国产大模型**，例如豆包、Kimi、DeepSeek，各自生成：

```
组合逻辑比较器代码
时序逻辑比较器代码
```

共 3 × 2 = 6 份代码。

这些代码不需要放入最终 `source` 文件夹，但需要在报告中比较：

有问题，尽管问 Thinking

ChatGPT 也可能会犯错。请核查重要信息。

这些代码不需要放入最终 `source` 文件夹，但需要在报告中比较：

代码规范性、功能正确性、设计思路贴合度、可仿真性。 数字逻辑大作业说明文档-20260403

第二部分：有符号数据最值输出模块

文件名：

```
signed_MM4.v
```

功能：输入 4 个有符号整数：

```verilog
num_a, num_b, num_c, num_d
```

输出：

```verilog
Max, Min
```

要求：

- `reset=1` 时清空内部寄存器和输出；
- `input_valid=1` 时输入数据有效；
- 输入、valid、Max/Min 都按时钟上升沿采样或更新；
- 要求在 `input_valid` 置零后的第一个周期结束前，`Max/Min` 更新完成；
- 需要实例化 `signed_compare_comb`。 数字逻辑大作业说明文档-20260403

第三部分：有符号数据流实时最值输出模块

文件名：

有问题，尽管问 Thinking

ChatGPT 也可能会犯错。请核查重要信息。

文件名：

```
signed_MMS.v
```

功能：串行输入一个有符号整数数据流，实时输出当前序列已经输入部分的最大值和最小值。

输入包括：

```verilog
clk, reset, data, input_valid, start
```

输出：

```verilog
Max, Min
```

关键要求：

- `reset=1` 时清零；
- `start=1` 表示新序列开始，需要把 `Max` 和 `Min` 置 0；
- `input_valid=1` 时，`data` 有效；
- 每个时钟周期更新当前已输入序列的最大值和最小值；
- `input_valid` 置零后的第一个周期结束前，`Max/Min` 必须更新完成；
- 需要实例化 `signed_compare_comb`。 数字逻辑大作业说明文档-20260403

报告注意点：

实验报告首页要放承诺书截图。如果使用 AI 辅助，必须在对应题目标注 AI 提供商和模型名称，并在附录中提供完整对话截图。 数字逻辑大作业说明文档-20260403

后续我会按这份文档的约束来帮你，尤其会注意这几点：

1. 不在比较器中直接使用比较运算符。

有问题，尽管问 Thinking

ChatGPT 也可能会犯错。请核查重要信息。

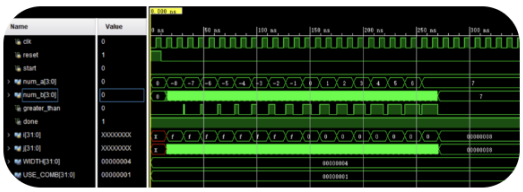
报告注意点：

实验报告首页要放承诺书截图。如果使用 AI 辅助，必须在对应题目标注 AI 提供商和模型名称，并在附录中提供完整对话截图。 数字逻辑大作业说明文档-20260403

后续我会按这份文档的约束来帮你，尤其会注意这几点：

1. 不在比较器中直接使用比较运算符。
2. 文件名、端口名、参数名保持和要求一致。
3. 代码要支持不同 `WIDTH` 参数。
4. 报告中区分“自己实现的正式代码”和“国产大模型对比代码”。
5. 仿真分析要对应 Tcl Console 输出和波形结果。

来源



在实验1-1的仿真中：
问题1：Tcl Console是什么？
问题2：如图是我写的程序输出的时序图，其中i和j最开始红色的部分是什么意思？

已思考 4s

问题1：**Tcl Console 是什么?**

在 Vivado 里，**Tcl Console** 是一个命令行窗口，主要有两个作用：

有问题，尽管问 Thinking

ChatGPT 也可能会犯错。请核查重要信息。

已思考 4s ›

问题1：**Tcl Console 是什么?**

在 Vivado 里，**Tcl Console** 是一个命令行窗口，主要有两个作用：

第一，可以输入 Tcl 命令控制 Vivado，例如运行仿真、综合、查看对象等。

第二，也是本实验更常用的作用：**显示 testbench 中 `$display`、`$monitor` 等语句打印出来的仿真结果。**

例如你的 testbench 里如果有类似：

```verilog
$display("num_a = %d, num_b = %d, greater_than = %d", num_a, num_b, greater_than);
```

那么仿真运行后，这些文字输出就会显示在 **Tcl Console** 窗口中。

所以报告中提到的"Tcl Console 输出结果"，通常指的就是 testbench 自动打印出的测试是否通过、当前输入输出结果、错误提示等文本信息。

问题2：**时序图中 i 和 j 最开始红色的部分是什么意思?**

图中 `i[31:0]` 和 `j[31:0]` 最开始显示为红色的 `x`，表示它们处于 **未知态 X。**

在 Verilog 仿真中，`reg`、`integer` 这类变量如果一开始没有被明确赋值，默认值就是 `x`，也就是 unknown，未知状态。

你的 `i` 和 `j` 很可能是 testbench 里面用于循环测试的变量，例如：

```verilog
integer i;
integer j;
```

在仿真刚开始的 0 ns 附近，如果还没有执行到给 i、j 赋值的语句，它们就会显示为红色 x。

+ 有问题，尽管问 Thinking

ChatGPT 也可能会犯错。请核查重要信息。

在仿真刚开始的 0 ns 附近，如果还没有执行到给 `i`、`j` 赋值的语句，它们就会显示为红色 `x`。

之后仿真开始执行循环，例如：

```verilog
for (i = 0; i < 8; i = i + 1) begin
    ...
end
```

此时 `i`、`j` 被赋予确定数值，就会从红色 `x` 变成绿色的具体数值。你图中后面 `i` 和 `j` 最终显示为 `00000008`，就是因为循环结束后变量停在了 8。

这里需要注意：`i` 和 `j` 一般只是 **testbench 内部的循环变量**，不是你的正式设计模块端口，也不是电路真实输出。它们开始为 `x` 通常不代表你的比较器设计有错误。

只要真正的输入输出，例如：

```verilog
num_a
num_b
greater_than
```

在测试过程中能按照预期变化，`greater_than` 能正确表示 `num_a > num_b`，那么实验 1-1 的组合逻辑仿真就是正常的。

在Verilog中，如果想把某个变量赋值为0且其共有WIDTH位（即该变量共有WIDTH个0）应当如何写?

有问题，尽管问

Thinking

ChatGPT 也可能会犯错。请核查重要信息。

在Verilog中，如果想把某个变量赋值为0且其共有WIDTH位（即该变量共有WIDTH个0），应当如何写?

在 Verilog 中，如果你想把一个变量赋值为全 0，且宽度为 `WIDTH` 位，可以使用 **位重复（replication）操作符**：

```verilog
variable = {WIDTH{1'b0}};
```

解释：

- `{WIDTH{…}}` 表示重复 `WIDTH` 次括号内的值；
- `1'b0` 表示一个比特的 0；
- 组合起来就是生成 `WIDTH` 位的全 0 值。

例如：

```verilog
reg [WIDTH-1:0] data;
...
data = {WIDTH{1'b0}};  // 将 data 赋值为全 0
```

如果你的变量是有符号数（`signed [WIDTH-1:0]`），同样适用：

```verilog
reg signed [WIDTH-1:0] num;
num = {WIDTH{1'b0}};   // num = 0
```

这是 Verilog 中标准且通用的做法，保证无论 `WIDTH` 多大，都能正确生成对应位宽的 0。

有问题，尽管问
Thinking
ChatGPT 也可能会犯错。请核查重要信息。

在实验1-2（时序逻辑实现有符号数比较器）中，代码模板提供了FSM的三种状态：IDLE、COMPARE和DONE，请你帮我理解一下这三种状态分别是什么意思？在一次比较已经完成之后，如果没有新的比较需求（start=0）且reset=0，系统应当保持在DONE状态还是应当回退到IDLE?

在实验 1-2（时序逻辑有符号比较器）中，FSM 的三种状态通常理解如下：

1. **IDLE**
   - 系统空闲状态，等待新的比较请求。
   - 当 `start=0` 时，FSM 会停留在 IDLE，输出 `done=0`。
   - 这是默认初始状态（reset 结束后进入 IDLE）。
2. **COMPARE**
   - 执行比较的状态。
   - 当 `start=1` 时，从 IDLE 转到 COMPARE，模块开始逐位比较 `num_a` 和 `num_b`。
   - 比较过程可能需要多个时钟周期（最多 `WIDTH + 1` 个周期）。
   - 在这个状态下，`greater_than` 输出逐步更新，但 `done` 保持 0。
3. **DONE**
   - 比较完成后的状态。
   - `done=1`，表示输出的 `greater_than` 结果已经有效。
   - 此状态保持直到：
     - **reset=1** → 回到 IDLE 并清零；
     - 或 **新的 start=1** → 触发下一次比较。

关于你的问题：一次比较完成后，如果没有新的比较请求（start=0）且 reset=0

- FSM **应当保持在 DONE 状态**，而不是自动回到 IDLE。
- 保持 DONE 状态的目的：
  1. 保证输出 `greater_than` 保持有效，直到外部需要新比较；
  2. 外部系统可以安全读取结果，而不会因为状态回到 IDLE 而丢失数据。
- 只有在 **reset=1** 或 **start=1** 时，FSM 才从 DONE 离开：

有问题，尽管问 Thinking

ChatGPT 也可能会犯错。请核查重要信息。

2. 外部系统可以安全读取结果，而不会因为状态回到 IDLE 而丢失数据。

- 只有在 **reset=1** 或 **start=1** 时，FSM 才从 DONE 离开：
  - reset → 回到 IDLE（清零）
  - start → 回到 COMPARE（开始新比较）

换句话说，FSM 状态流可以概括为：

```
IDLE --(start=1)--> COMPARE --(比较完成)--> DONE --(start=1)--> COMPARE
                                  |
                                  --(reset=1)--> IDLE
```

DONE 是 **输出锁存状态**，保证结果稳定，直到下一次触发或复位。

如果你愿意，我可以画一张简单的状态机图，标出每个状态及条件转换，更直观理解。

你希望我画吗?

如果按照你所说的状态转移逻辑，当检测到start=1时，系统从IDLE或DONE转移到COMPARE状态，并从下一个时钟周期开始逐位比较，最坏情况可能需要WIDTH+1个时钟周期。我希望在IDLE和DONE中就完成MSB的比较，也即，当检测到start=1时，在同一个时钟周期内就比较两个数的MSB，如果不同，直接输出结果，状态转移到DONE，结束比较，如果不同再转移到COMPARE状态逐位比较后几位，这样，最坏情况也只需要WIDTH个时钟周期就能输出结果，可以提高效率。这样是合理的吗?

已思考 12s

你的想法**在硬件设计上是合理的**，而且确实是一种常见的时序优化：把第一步比较，也就是符号位 `MSB` 的比较，提前放在 `IDLE` 或 `DONE` 状态中，在检测到 `start=1` 的那个时钟上升沿就完成。

有问题，尽管问 Thinking

ChatGPT 也可能会犯错。请核查重要信息。

你的想法**在硬件设计上是合理的**，而且确实是一种常见的时序优化：把第一步比较，也就是符号位 `MSB` 的比较，提前放在 `IDLE` 或 `DONE` 状态中，在检测到 `start=1` 的那个时钟上升沿就完成。

对于有符号数比较，最高位是符号位：

verilog

```
num_a[WIDTH-1]
num_b[WIDTH-1]
```

如果符号位不同，就可以立刻判断大小：

```
num_a 符号位 = 0，num_b 符号位 = 1  → num_a 为正，num_b 为负，num_a > num_b
num_a 符号位 = 1，num_b 符号位 = 0  → num_a 为负，num_b 为正，num_a < num_b
```

这种情况下不需要再比较后面的数值位，可以直接进入 `DONE` 状态。

如果符号位相同，才需要进入 `COMPARE` 状态，从次高位 `WIDTH-2` 开始逐位比较。

因此，你的状态转移可以理解为：

```
IDLE / DONE
    |
    | start = 1
    v
先比较 MSB
    |
    | MSB 不同
    v
DONE，直接输出结果

先比较 MSB
    |
    | MSB 相同
    v
COMPARE，继续比较 WIDTH-2 到 0 位
```

有问题，尽管问 Thinking

ChatGPT 也可能会犯错。请核查重要信息。

这样设计后，最坏情况确实可以从原来的大约 `WIDTH + 1` 个周期优化到 `WIDTH` 个周期以内。

例如 `WIDTH=4`，比较过程可以是：

```
第 1 个周期：检测 start，并比较 bit[3]，即符号位
第 2 个周期：比较 bit[2]
第 3 个周期：比较 bit[1]
第 4 个周期：比较 bit[0]
```

所以最坏情况 4 个周期完成，优于文档中要求的 `WIDTH + 1` 周期上限。

不过有一个细节要注意：**在 DONE 状态下重新检测到 start=1 时，done 是否需要先被清零。**

文档中说：

```
done 在 reset 置高或新的 start 到来时被清零
```

所以最严格地理解，新的 `start` 到来时，`done` 应该被清零，表示上一轮结果失效，开始新一轮比较。

但如果你在同一个时钟上升沿里既检测到 `start=1`，又发现 MSB 不同，并立即令 `done=1`，那么波形上可能看起来 `done` 没有经历一个明显的低电平，尤其是系统原本就在 `DONE` 状态时。这在逻辑上可以解释为“新比较瞬间完成”，但可能和 testbench 对 `done` 清零过程的期待不完全一致。

所以更稳妥的处理方式是：

- 从 `IDLE` 状态收到 `start=1` 时，可以直接比较 MSB；
- 从 `DONE` 状态收到新的 `start=1` 时，也可以直接开始新比较，但要确认 testbench 是否允许 `done` 在同一周期重新变为 1；
- 如果担心评测脚本严格检查“新 start 到来时 done 必须清零”，可以让 `done` 至少低一个周期，再进入 `DONE`。

有问题，尽管问
Thinking
ChatGPT 也可能会犯错。请核查重要信息。

如果从实验要求和性能要求来看，你的优化方案是**完全合理的**，因为它没有违反"不能直接使用比较运算符"的限制，而且更体现了逐位比较的硬件思想。

比较推荐的写法思路是：

verilog

```verilog
if (start) begin
    done <= 1'b0;

    if (num_a[WIDTH-1] != num_b[WIDTH-1]) begin
        greater_than <= (~num_a[WIDTH-1]) & num_b[WIDTH-1];
        done <= 1'b1;
        state <= DONE;
    end
    else begin
        bit_index <= WIDTH - 2;
        state <= COMPARE;
    end
end
```

这里本质上就是：**start 到来时先比较符号位，符号位能决定结果就直接完成，不能决定再进入逐位比较。**

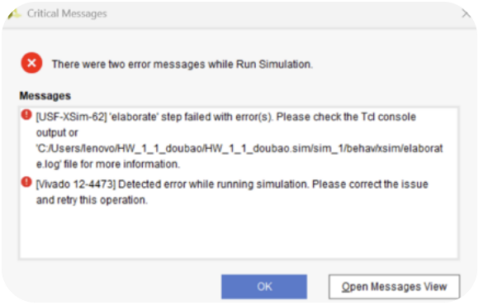
注意你原话里有一句应该是笔误：

> 如果不同，直接输出结果，状态转移到 DONE，结束比较，如果不同再转移到 COMPARE 状态逐位比较后几位

第二个"如果不同"应当改成：

> 如果**相同**，再转移到 `COMPARE` 状态逐位比较后几位。

Critical Messages
有问题，尽管问
Thinking
ChatGPT 也可能会犯错。请核查重要信息。



下面是豆包写的1-1实验的代码，我知道其中存在wire型变量在always语句中赋值的问题，我已经修改了这个问题（新定义一个reg型变量用于在always中临时存储结果，再在always语句外用assign将结果赋给greater_than），但仿真时仍然报出图中的错误，这是为什么？

```
module signed_compare_comb #(
    parameter WIDTH = 4 // Bit-width of operands
)(
    input wire [WIDTH-1:0] num_a, // (signed WIDTH-bit)
    input wire [WIDTH-1:0] num_b, // (signed WIDTH-bit)
    output wire greater_than      // 1: num_a > num_b, 0: else
);
```

展开

已思考几秒 >

主要原因很可能不是 `wire` 在 `always` 中赋值，而是这一句：

```verilog
break;
```

在 **Verilog** 中，`for` 循环里通常不能使用 `break`。`break` 是 **SystemVerilog** 支持的语法，但本实验一般要求写的是 Verilog 文件 `.v`，Vivado 按 Verilog 语法编译时就会报语法错误，所以会出现你图中这种：

+ 有问题，尽管问 Thinking

ChatGPT 也可能会犯错。请核查重要信息。

```
elaborate step failed with error(s)
```

也就是说，仿真还没真正跑起来，在 elaboration 阶段代码就没有通过编译/展开。

你现在的代码中有两个问题：

1. `greater_than` 是 `output wire`，不能在 `always` 中赋值

原代码里：

verilog
```
output wire greater_than
```

但在 `always @(*)` 中写了：

verilog
```
greater_than = sign_case | (~sign_different & same_sign_greater);
```

这是不允许的。你已经改成用 `reg` 临时变量再 `assign`，这个方向是对的。

---

2. `break` 不能用于普通 Verilog

原代码：

verilog
```
for (i = WIDTH-1; i >= 0; i = i - 1) begin
    if (num_a[i] != num_b[i]) begin
        same_sign_greater = num_a[i];
        break;
    end
```

有问题，尽管问 Thinking

ChatGPT 也可能会犯错。请核查重要信息。

2. `break` 不能用于普通 Verilog

原代码：

verilog

```verilog
for (i = WIDTH-1; i >= 0; i = i - 1) begin
    if (num_a[i] != num_b[i]) begin
        same_sign_greater = num_a[i];
        break;
    end
end
```

这里的目的本来是：从高位到低位扫描，找到第一位不同的地方后就停止。

但是 Verilog 里不能这样写，所以需要自己定义一个标志变量，例如：

verilog

```verilog
reg found_diff;
```

用 `found_diff` 表示“是否已经找到更高位的不同位”。

可以改成下面这种写法：

verilog

```verilog
module signed_compare_comb #(
    parameter WIDTH = 4
)(
    input wire [WIDTH-1:0] num_a,
    input wire [WIDTH-1:0] num_b,
    output wire greater_than
);

    wire sign_a;
    wire sign_b;
```

有问题，尽管问 Thinking

ChatGPT 也可能会犯错。请核查重要信息。

verilog

```verilog
    wire sign_different;
    wire sign_case;

    reg same_sign_greater;
    reg result;
    reg found_diff;
    integer i;

    assign sign_a = num_a[WIDTH-1];
    assign sign_b = num_b[WIDTH-1];
    assign sign_different = sign_a ^ sign_b;

    // 符号位不同：num_a 为正、num_b 为负时，num_a > num_b
    assign sign_case = sign_different & (~sign_a);

    always @(*) begin
        same_sign_greater = 1'b0;
        found_diff = 1'b0;

        // 符号位相同时，从次高位开始逐位比较即可
        for (i = WIDTH-2; i >= 0; i = i - 1) begin
            if ((!found_diff) && (num_a[i] != num_b[i])) begin
                same_sign_greater = num_a[i];
                found_diff = 1'b1;
            end
        end

        result = sign_case | ((~sign_different) & same_sign_greater);
    end

    assign greater_than = result;

endmodule
```

这里的核心思想是：

verilog

有问题，尽管问 Thinking

ChatGPT 也可能会犯错。请核查重要信息。

这里的核心思想是：

```verilog
found_diff = 0
```

表示还没有找到高位不同的地方；一旦找到第一位不同的位，就令：

```verilog
found_diff = 1
```

之后即使低位不同，也不再更新 `same_sign_greater`。这样就等价于原来 `break` 的效果。

另外建议你打开 Vivado 下面的 **Tcl Console**，真正的语法错误一般会显示得更具体，可能会有类似：

```
syntax error near "break"
```

或者：

```
break statement not supported
```

你截图里的弹窗只是总提示，真正定位错误要看 Tcl Console 里的详细报错行号。

有问题，尽管问 Thinking

ChatGPT 也可能会犯错。请核查重要信息。